

USAGE

VisionEngine Usage

Provider Configuration

Set API keys via environment variables:

```
VISION_PROVIDER=auto          # "openai", "anthropic", "gemini",  
                               "qwen", "ollama", "auto"  
OPENAI_API_KEY=sk-...  
ANTHROPIC_API_KEY=sk-ant-...  
GOOGLE_API_KEY=AIza...  
QWEN_API_KEY=sk-...  
VISION_TIMEOUT_SECS=30       # per-request timeout (default: 30)  
VISION_MAX_IMAGE_SIZE=20971520 # 20 MB default
```

Provider-specific model overrides:

```
OPENAI_MODEL=gpt-4o  
ANTHROPIC_MODEL=claude-sonnet-4-20250514  
GEMINI_MODEL=gemini-2.0-flash  
QWEN_MODEL=qwen-vl-max
```

When `VISION_PROVIDER=auto`, the `FallbackProvider` tries providers in order based on available API keys.

Basic Usage

```
import (  
    "digital.vasic.visionengine/pkg/config"  
    "digital.vasic.visionengine/pkg/llmvision"  
)  
  
cfg := config.FromEnv()  
provider, err :=  
    llmvision.NewGeminiProvider(llmvision.ProviderConfig{  
        APIKey:    cfg.GoogleAPIKey,  
        Model:     cfg.GeminiModel,  
        MaxTokens: 4096,  
    })
```

```

})

result, err := provider.AnalyzeImage(ctx, screenshotBytes,
    "Describe this screen")

```

Fallback Chain

```

providers := []llmvision.VisionProvider{gemini, openai, anthropic}
fallback, err := llmvision.NewFallbackProvider(providers...)
result, err := fallback.AnalyzeImage(ctx, image, prompt)
// Tries gemini first, then openai, then anthropic

```

GPU Management

When using Ollama locally, VisionEngine does not manage GPU allocation directly. Instead, the HelixQA orchestrator calls helper functions:

- **FreeGPU:** Stop Ollama service to release VRAM for other workloads (e.g., NVIDIA inference containers)
- **RestoreOllama:** Restart Ollama after the GPU-intensive workload completes

These are shell-level operations, typically:

```

# Free GPU (stop Ollama)
systemctl stop ollama
# or: pkill ollama

# Restore Ollama
systemctl start ollama
# or: nohup ollama serve &

```

Remote Deployment

Deploy and manage Ollama on a remote GPU host via SSH:

```

import "digital.vasic.visionengine/pkg/remote"

deployer := remote.NewDeployer(remote.Config{
    Host:  "thinker.local",
    User:  "milosvasic",
    Model: "llava:7b",      // default
    Port:  22,              // SSH port
    OllamaPort: 11434,      // Ollama API port
})

// Full deployment: install + start + pull model

```

```

endpoint, err := deployer.EnsureReady(ctx)
// endpoint = "http://thinker.local:11434"

// Check status without making changes
status := deployer.Status(ctx)
// status.OllamaInstalled, status.OllamaRunning,
// status.ModelAvailable

```

Vision Slots (Multi-Device)

When testing multiple devices simultaneously, use VisionSlots to prevent contention:

```

slot := &remote.VisionSlot{
    ID:          "androidtv-192.168.0.134:5555",
    Platform:    "androidtv",
    Device:      "192.168.0.134:5555",
    Endpoint:    "http://thinker.local:11434",
}

slot.Lock()
defer slot.Unlock()
// Make vision call -- serialized per device
result, err := provider.AnalyzeImage(ctx, screenshot, prompt)

```

Navigation Graph

Build a screen navigation graph during QA exploration:

```

import "digital.vasic.visionengine/pkg/graph"

g := graph.NewNavigationGraph()
loginID := g.AddScreen(analyzer.ScreenIdentity{Name: "Login",
    Category: "auth"})
homeID := g.AddScreen(analyzer.ScreenIdentity{Name: "Home",
    Category: "main"})
g.AddTransition(loginID, homeID, analyzer.Action{Type: "click",
    Target: "Sign In"})

// Find shortest path
path, err := g.PathTo(homeID)

// Track exploration progress
coverage := g.Coverage() // 0.0 to 1.0
unvisited := g.UnvisitedScreens() // guide curiosity phase

```

Build Tags

Default: stubs for OpenCV, LLM providers work

```
go build ./...
```

```
go test ./...
```

With OpenCV support (requires GoCV + OpenCV installed)

```
go build -tags vision ./...
```

```
go test -tags vision ./...
```